```
        x = torch.relu(x)
        x = self.conv2(x)
        x = torch.relu(x)
        with torch.no_grad():
            self.conv_map = x.clone()
3
        _,_,cH,cW = x.shape
        xcoords = torch.arange(cW).repeat(cH,1).float() / cW
        ycoords =
torch.arange(cH).repeat(cW,1).transpose(1,0).float() / cH
        spatial_coords = torch.stack([xcoords,ycoords],dim=0)
        spatial_coords = spatial_coords.unsqueeze(dim=0)
        spatial_coords = spatial_coords.repeat(N,1,1,1)
        x = torch.cat([x,spatial_coords],dim=1)
        x = x.permute(0,2,3,1)
        x = x.flatten(1,2)
        K = rearrange(self.k_proj(x), "b n (head d) -> b head n d",
            head=self.n_heads)
        K = self.k_norm(K)
        Q = rearrange(self.q_proj(x), "b n (head d) -> b head n d",
            head=self.n_heads)
        Q = self.q_norm(Q)
        V = rearrange(self.v_proj(x), "b n (head d) -> b head n d",
            head=self.n_heads)
        V = self.v_norm(V)
        A = torch.nn.functional.elu(self.q_lin(Q) + self.k_lin(K))
4
        A = self.a_lin(A)
        A = torch.nn.functional.softmax(A,dim=3)
        with torch.no_grad():
            self.att_map = A.clone()
5
        E = torch.einsum('bhfc,bhcd->bhfd',A,V)
6
        E = rearrange(E, 'b head n d -> b n (head d)')
7
        E = self.linear1(E)
        E = torch.relu(E)
        E = self.norm1(E)
        E = E.max(dim=1)[0]
        y = self.linear2(E)
        y = torch.nn.functional.elu(y)
        return y
```

- ***1*** We use 1 × 1 convolutions to preserve the spatial organization of the objects in the grid.
- ***2*** Sets up linear layers for additive attention
- ***3*** Saves a copy of the post-convolved input for later visualization